

CSC 4553

Fall 2026 - Day 13

Admin

- Programming assignment 2 (schedfit) due Monday, 18:00
- Lab due Friday, 18:00
- Quiz due Monday, 18:00
- Exam regrade request deadline is Friday, 18:00

Primitives

Questions to consider

- How do synchronization solutions compare in implementation / function?
- What is priority inversion, and how does priority inheritance fix it?
- When would you choose a counting semaphore versus a binary semaphore or mutex, and what distinguishes them?

Hardware solutions: atomicity

- Recall the naïve `bool` lock: both threads read `lock == FALSE` in the gap before either one set it
- Fix: let the hardware do the read *and* the write as **one indivisible step**, an **atomic instruction**
 - No other CPU can touch that memory location partway through
- Why not just disable interrupts? (this was the original solution)
 - Only stops preemption on *this* CPU; other cores keep running

Atomic instructions

- Two classic ones:
 - `test_and_set(&x)`: set `x` to true and return its *old* value
 - `compare_and_swap(&x, expected, new)`: only if `x == expected`, set it to `new`; return the old value
 - Real APIs differ: C11 `atomic_compare_exchange_strong()` returns a bool (did the swap happen?) and writes the old value back into `expected`
 - Same information either way: `old value == expected` means the swap happened
 - Either way, one instruction both checks “was it free?” and claims it
- Limit: works on a single word of memory, not on big operations
 - An “atomic B-tree update” instruction? No.

Does this solution meet the requirements?

```
1 bool test_and_set(bool *lock){
2     bool ret = *lock;
3     *lock = TRUE;
4     return ret;
5 }
6
7 lock = FALSE;
8 do{
9     while(test_and_set(&lock));
10    // CRITICAL SECTION
11    lock = FALSE;
12 }while(true);
```

- Mutual exclusion?
 - Yes
- Progress?
 - Yes: when the lock is released, a waiting process will acquire it
- Bounded waiting?
 - No: an unlucky process could be endlessly passed over based on the scheduler's decisions

Mutex locks

- Atomic instruction-based solutions are complicated and generally not used by application programmers
- Any solution to the critical section problem requires a lock
 - Peterson's solution is an example (in software)
 - `test_and_set()` / `compare_and_swap()` are hardware implementations
- Simplest commonly used solution is **mutex** lock
 - Boolean variable indicating if lock is available or not

Mutex locks (cont'd)

- Protect a critical section by
 - First **acquire()** a lock
 - Then **release()** the lock
- What's going on under the hood?

```
1 void lock(lock_t *lock) {  
2     while (test_and_set(&lock->flag))  
3         ; // spin-wait (do nothing)  
4 }
```

→ **test_and_set()** or **compare_and_swap()**

Mutex locks (cont'd)

- Calls to `acquire()` and `release()` must be **atomic**
- But this solution requires **busy waiting**
 - This type of mutex is therefore often called a **spinlock**
- There are also mutexes that block instead of busy-waiting (e.g., those built on futexes), and some that do both
- Note: **mutexes are owned by the thread that locks them**. *Only the thread that acquired the mutex can release it*

Futex: fast path, slow path

- Blocking is safe but costly (a syscall every time); real systems want the common case to be cheap
- **Futex** (**F**ast **U**userspace mu**TEX**, Linux 2002) makes that possible
- Key idea: most lock acquisitions are **uncontended**
 - Put the lock variable in **user-space memory**
 - Try to grab it with a single atomic instruction (e.g., **cmpxchg**)
 - If you win → done. No syscall, no kernel involvement (~10 ns)
- Only on **contention** do you call into the kernel via the **futex()** syscall
 - Kernel adds you to a wait queue and puts you to sleep
 - Releaser, if there are waiters, calls **futex()** to wake one up

Futex (cont'd)

- This is why `pthread_mutex_t` on Linux is cheap when uncontended — it's built on futexes
- Same story for `std::mutex` in C++ and most modern language runtimes
- Question: why can't you build a *blocking* mutex purely in user space?
 - Blocking an OS thread — taking it off the run queue — is a kernel operation by definition
 - Only the kernel can decide which thread runs on a CPU
 - Futex is the **minimum-syscall** way to ask the kernel for that service

Hidden costs of locking

In 1997, NASA's Mars Pathfinder started having mysterious resets on Mars. From Earth, engineers saw the system repeatedly rebooting for no obvious reason. Not ideal when your computer is ~190 million km away.

The cause: **priority inversion**.

- Low-priority task holds a mutex
- High-priority task needs it, so it blocks
- Medium-priority task (doesn't need the mutex) keeps preempting the low-priority one
- Low never runs, so it never releases; high is starved
- A watchdog sees the stall as a fault and resets the system. Over and over.

Priority inversion

- Any ideas on how to fix?
 - **Priority inheritance**: low priority processes are temporarily given priorities that match those waiting on them

What isn't clear?

Comments? Thoughts?

Semaphores

- Synchronization tool that provides more sophisticated ways (than mutex locks) for processes to synchronize their activities
- Semaphore **S**: integer variable
- Can only be accessed via two atomic operations
 - **wait()** and **signal()**
 - Originally called **P()** and **V()** (because, Dutch)
- Do not have ownership, any thread can signal (increment) or wait (decrement)

```
1 wait(S) {  
2     while (S <= 0)  
3         ; // busy  
4     S--;  
5 }  
6  
7 signal(S) {  
8     S++;  
9 }
```

Semaphores (cont'd)

- **Binary semaphore**: integer value can range only between 0 and 1
 - Functionally similar to a mutex, but without ownership — any thread can signal it
- **Counting semaphore**: integer value can range over an unrestricted domain
 - When would this make sense?
 - Protect a finite set of resources rather than a single one (e.g., you have a pool of shared db connections)
 - Initialize **S** to the number of resources
 - Decrement **S** each time someone grabs a resource, increment when they release

```
1 do{  
2   wait(mutex);  
3   //CS  
4   signal(mutex);  
5 }while(TRUE);
```

Enforce the order

Semaphore can also be used to impose order...

Three threads each loop forever: one prints **A**, one prints **B**, one prints **C**.

Using **only semaphores**, guarantee the output is always **ABCABCABC...**

- How many semaphores do you need?
- What are their initial values?
- Where do the **wait()** and **signal()** calls go?

Blocking semaphores

- We add `sleep()` and `wakeup()` to the underlying implementation
- Call to wait, and semaphore not available, process is blocked with `sleep()` (wait state) and placed on a waiting queue
- Blocked processes are notified of an available semaphore by the `wakeup()` operation
 - goes from waiting to ready
 - Able to achieve bounded wait and progress with FIFO queue

```
1 wait(semaphore *S) {  
2     S->value--;  
3     if (S->value < 0) {  
4         add process to S->list;  
5         sleep();  
6     }  
7 }  
8  
9 signal(semaphore *S) {  
10    S->value++;  
11    if (S->value <= 0) {  
12        remove P from S->list;  
13        wakeup(P);  
14    }  
15 }
```

Mutexes vs. semaphores

- Mutexes are generally lighter weight / faster
- Semaphores can support multiple instances
- Semaphores don't have the **ownership limitations** of mutexes

Choose your primitive

- For each, choose between: atomic instructions; spin lock mutexes; blocking (futex-based) mutexes; semaphores
- Scenario 1: You have a pool of database connections, and you need to limit the number of threads that can access these connections simultaneously.
 - Counting semaphore
- Scenario 2: You need to increment a shared counter frequently, and the operation is very quick.
 - Atomic instruction
- Scenario 3: A kernel data structure protected by a lock held for ~50 CPU cycles.
 - Spin lock mutex

Choose your primitive (cont'd)

- For each, choose between: atomic instructions; spin lock mutexes; blocking (futex-based) mutexes; semaphores
- Scenario 4: A user-space cache entry lock held for $\sim 100\ \mu\text{s}$ while computing a result.
 - Blocking mutex (e.g., `pthread_mutex_t`, built on a futex)
- Scenario 5: A file lock held for several seconds while the file is being edited.
 - Blocking mutex
- Scenario 6: A lock on a single-CPU system where only one thread can run at a time.
 - Blocking mutex: never spin on a uniprocessor

Beware: mistakes are easy to make

- Incorrect use of semaphore operations
 - wrong order: `signal(mutex) ... wait(mutex)`
 - repeating: `wait(mutex) ... wait(mutex)`
 - Omitting of `wait(mutex)` and/or `signal(mutex)`
- Be careful

Condition variables

- Paired with a mutex to let threads wait for a condition rather than spin
- `x.wait()`: calling thread releases the mutex and suspends until signaled; it reacquires the mutex before returning
- `x.signal()`: wakes one waiter (if any); no effect if no waiters
- `x.broadcast()`: wake *all* waiters — useful when multiple threads may now be able to proceed
- **Not like a semaphore**: a semaphore *remembers* a signal (the count stays incremented, so a later `wait()` goes right through); a CV signal with no one waiting is simply **lost**
- Good fit for producer-consumer and any case where threads must sleep until a resource / condition is ready

What isn't clear?

Comments? Thoughts?